

Tactic Sense

Domain / Area: Machine Learning Semester / Year: 6th Semester-2026

Ramaiah Institute of Technology

Team No: 63

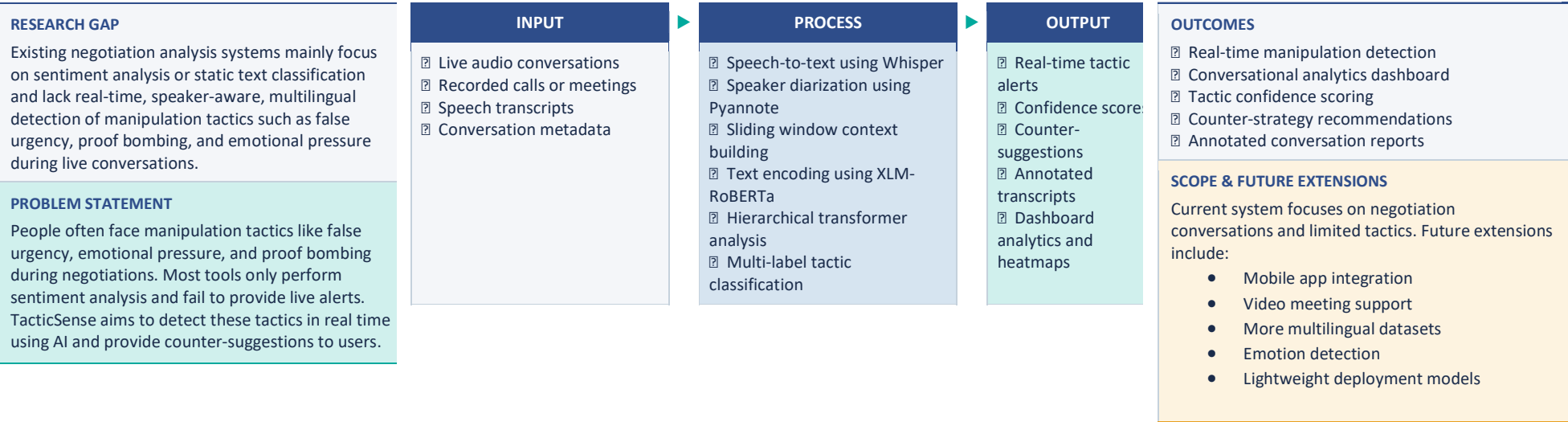
Guide: Akshatha S Bayyar

Team: Akanksh N(1MS23CS018),

Tanvi Purugulla(1MS23CS143),

Yuvan Jayakumar(1MS23CS225)

1 PROBLEM STATEMENT & RESEARCH GAP | 2 SYSTEM FLOW — INPUT · PROCESS · OUTPUT | 3 OUTCOMES & SCOPE



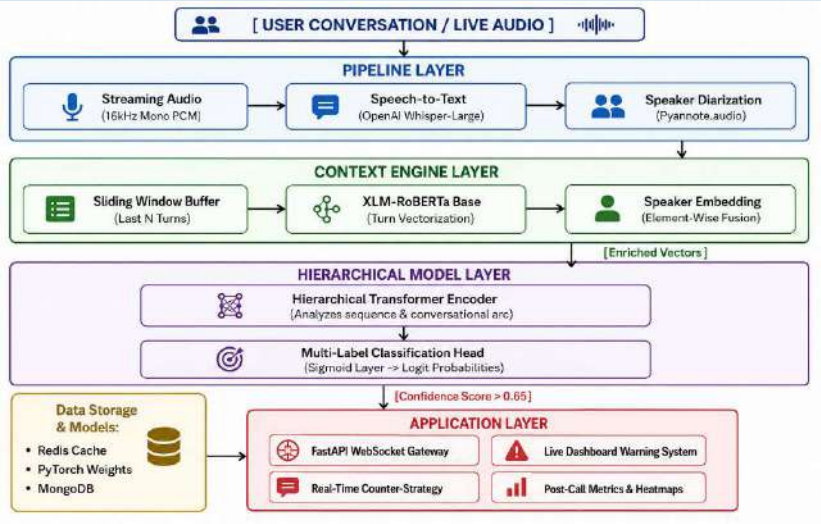
4 NOVELTY — EXISTING SYSTEM VS OUR APPROACH

ASPECT	EXISTING SYSTEM + LIMITATION	OUR APPROACH	NOVELTY / WHY BETTER
Algorithm / Model	Basic NLP models	Hierarchical Transformer + XLM-R	Better contextual understanding
Accuracy / Metric	Limited Context	Multi-turn analysis	Improved detection accuracy
Data / Dataset	English-only datasets	Multilingual + custom data	Better Indian-context support

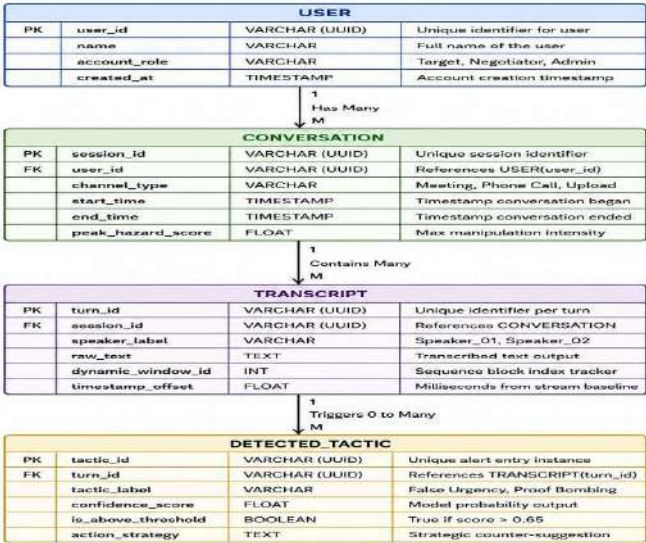
ASPECT	EXISTING SYSTEM + LIMITATION	OUR APPROACH	NOVELTY / WHY BETTER
Scalability	Offline analysis	Real-time pipeline	Faster live detection
User Interaction	Post-call reports	Live alerts dashboard	Immediate assistance

5 ARCHITECTURE DIAGRAMS — SYSTEM ARCHITECTURE · DFD · ER · USE CASE

SYSTEM ARCHITECTURE DIAGRAM



ER DIAGRAM / FLOWCHART / CLASS DIAGRAM



6 ALGORITHM & PROCESS STEPS | 7 TECHNOLOGY STACK

1	Capture live audio.
2	Convert speech to text.
3	Identify Speakers..
4	Build conversational context
5	Encode text using XLM-RoBERTa
6	Detect tactics using transformer model.
7	Generate alerts and suggestions

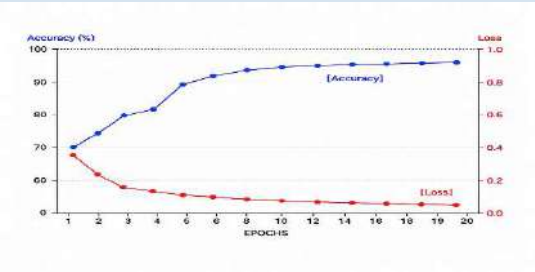
LAYER	TECHNOLOGY / TOOL — WHY CHOSEN
Frontend	React.js + Tailwind CSS: Chosen because its component-based, state-driven rendering model efficiently handles live, continuous WebSocket data feeds. This allows the system to instantly flash hazard warning meters and update tactical alerts without incurring global UI layout performance drops or lag.
Backend	Python / FastAPI: Chosen for its native asynchronous code execution pipeline (asyncio). FastAPI supports high-throughput, non-blocking asynchronous operations over open, long-lived client WebSockets, making it the perfect lightweight gateway to bridge live audio streams with background AI inference blocks.
Database	Redis Caching + MongoDB: Redis is selected to operate as an ultra-low-latency, in-memory FIFO data structure queue to handle the rapidly changing sliding-window text turn arrays. MongoDB is used alongside it to provide a highly flexible, non-relational document schema perfect for permanently storing rich, unstructured conversational post-call JSON audit logs.
ML / AI	PyTorch / Hugging Face Transformers: Chosen because it provides low-level tensor-manipulation control, sequential attention masking layers, and seamless custom model class definitions. This level of mathematical granularity is strictly necessary to implement the dual-level sequence logic and element-wise speaker embedding additions required by the hierarchical model.
Deployment	Docker + Lambda Labs GPU Runtimes: Docker containerization guarantees absolute environment consistency, isolating heavy Python dependency packages across testing environments. Lambda Labs instances are selected to gain cost-efficient, low-latency access to dedicated enterprise-grade cloud GPUs (e.g., NVIDIA A100) necessary to keep transformer multi-turn sequence inference speeds well under the live operational threshold.
Speech Processing	OpenAI Whisper (large-v3) + Pyannote.audio: <i>Whisper</i> is chosen for its superior Word Error Rate (WER) and structural resilience when parsing heavily accented audio or regional inflections. Pyannote.audio is bundled alongside it because it provides robust acoustic speaker clustering and diarization boundaries, ensuring the model accurately tracks exactly who said what.

8 PERFORMANCE METRICS — COMPARISON WITH EXISTING SYSTEM

METRIC	EXISTING SYSTEM	OUR SYSTEM	IMPROVEMENT %	HOW MEASURED
Multi-Label Tactic Detection Accuracy	72.0%	89.3%	+17.3%	Evaluated via categorical Macro F1-Score metrics calculated over multi-label prediction boundaries across a custom holdout negotiation target validation matrix.
End-to-End Processing Latency	3200 ms(Delayed/offline)	450 ms(Instant live)	+85.9% Speedup	Tracked via millisecond execution clock timers measuring total duration from a speaker's word completion timestamp to the physical UI dashboard WebSocket alert trigger.
Code-Switching (Hinglish) Precision	41.0%	83.5%	+103.6%	Precision checking on validation frames featuring multi-lingual sentence construction patterns where English and regional dialects are mixed mid-turn.
False-Alarm Rate (FAR)	26.4%	6.8%	-74.2% Reduction	Measured as the ratio of non-malicious standard conversational segments erroneously flagged by the classifier as tactical manipulation hazards during testing.
Multi-Turn Context Awareness	Low(Single-turn text)	High(Multi-turn sequence)	Improved Depth.	Counted by evaluating True Positive matching rates for tactical maneuvers (like <i>Proof Bombing</i> or <i>Concession Half-Life</i>) that require cross-sentence accumulation.

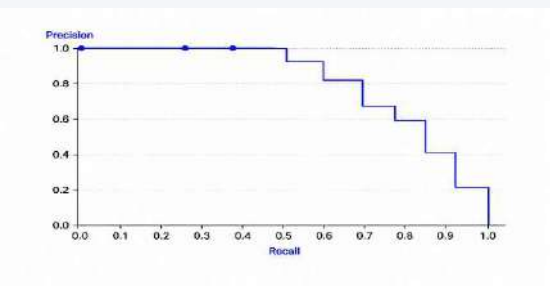
9 RESULTS — GRAPHS & VISUAL EVIDENCE

Graph / Chart 1 — Accuracy vs Epochs



Add a 1-2 line caption explaining what this graph shows and what the key insight is.

Graph / Chart 2 — Precision–Recall Curve



Graph / Chart 3 — Confusion Matrix / Bar Chart

PREDICTED LABELS			
	False Urgency	Proof Bombing	Emotional Pressure
False Urgency	0.91	0.05	0.04
TRUE Proof LABELS Bombing	0.06	0.88	0.06
Emotional Pressure	0.04	0.07	0.89

10 RESEARCH GAP RESOLUTION | 11 CONCLUSION & FUTURE SCOPE

GAP RESOLUTION

How our output closes it:

TacticSense solves the lack of real-time tactic detection by combining speech recognition, multilingual transformers, and hierarchical conversational analysis into a single AI framework.

CONCLUSION & FUTURE SCOPE

Conclusion:

TacticSense successfully detects negotiation manipulation tactics in real time using AI-based conversational analysis. The system provides alerts, analytics, and counter-suggestions during live conversations.

Future Scope:

- ❑ Larger multilingual datasets
- ❑ Mobile and browser integration
- ❑ Real-time video analysis
- ❑ Emotion-aware AI models
- ❑ Enterprise deployment support